

33 Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по «Алгоритмам и структурам данных»

Базовые задачи

Выполнил:

Студент группы Р3206

Косогова М. А.

Преподаватели:

Косяков М.С.

Тараканов Д. С.

Санкт-Петербург

2026

Задача №Е «Коровы в стойла»

```
#include <fstream>
```

```
#include <iostream>
```

```
int main() {
```

```
    int N;
```

```
    int K;
```

```
    std::ifstream input("input.txt");
```

```
    if (input.is_open()) {
```

```
        if (input >> N && input >> K && N > 2 && N < 100000 && K > 1 && K < N) {
```

```
            int* stalls = new int[N];
```

```
            for (int i = 0; i < N; i++) {
```

```
                input >> stalls[i];
```

```
            }
```

```
            int res_dis = 1;
```

```
            int min_dis = 2;
```

```
            int max_dis = stalls[N - 1] - stalls[0] - (K - 2);
```

```
            while (min_dis <= max_dis) {
```

```
                int med_dis = min_dis + (max_dis - min_dis) / 2;
```

```
                int cows = 1;
```

```
                int last = stalls[0];
```

```
                int i = 1;
```

```
                while (cows < K && i < N) {
```

```
                    if (stalls[i] - last >= med_dis) {
```

```
                        cows++;
```

```
                        last = stalls[i];
```

```
                    }
```

```
                    i++;
```

```
                }
```

```
            if (cows == K) {
```

```
                res_dis = med_dis;
```

```
                min_dis = med_dis + 1;
```

```

    } else {
        max_dis = med_dis - 1;
    }
}

delete[] stalls;
std::ofstream output("output.txt");
if (output.is_open()) {
    output << res_dis;
    output.close();
}
} else {
    std::cout << "N K read error!" << std::endl;
}
input.close();
} else {
    std::cout << "input input.txt not found!" << std::endl;
}
return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Считываем количество стойл и коров, координаты стойл;
- 2) Задаем границы бинарного поиска по минимальному расстоянию между коровами;
- 3) Для текущего среднего расстояния жадно пытаемся разместить всех коров слева направо;
- 4) Если удастся разместить всех коров, запоминаем ответ и пробуем увеличить расстояние, иначе уменьшаем верхнюю границу;
- 5) Записываем максимальное найденное минимальное расстояние, при котором возможно размещение всех коров, в выходной файл.

Задача №F «Число»

```

#include <algorithm>
#include <fstream>
#include <iostream>

```

```

#include <string>

bool compare_parts(const std::string& s1, const std::string& s2) {
    return ((s1 + s2) > (s2 + s1));
}

int main() {
    std::ifstream input("number.in");
    if (input.is_open()) {
        std::string parts[100];
        int n = 0;
        while (input >> parts[n] && n < 100) {
            n++;
        }
        input.close();
        sort(parts, parts + n, compare_parts);
        std::ofstream output("number.out");
        if (output.is_open()) {
            for (int i = 0; i < n; i++) {
                output << parts[i];
            }
            output.close();
        }
    } else {
        std::cout << "input number.in not found!" << std::endl;
    }
    return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Считываем все числовые строки из входного файла в массив;
- 2) Сортируем строки с помощью компаратора: строка a предшествует строке b , если $a + b > b + a$;

- 3) Последовательно записываем отсортированные строки в выходной файл, формируя максимально возможное число.

Задача №G «Кошмар в замке»

```
#include <algorithm>
```

```
#include <fstream>
```

```
#include <iostream>
```

```
#include <string>
```

```
int weights[26];
```

```
bool compare_weights(const char& char1, const char& char2) {
```

```
    return weights[char1 - 'a'] > weights[char2 - 'a'];
```

```
}
```

```
int main() {
```

```
    std::ifstream input("aurora.in");
```

```
    if (input.is_open()) {
```

```
        std::string s;
```

```
        input >> s;
```

```
        for (int i = 0; i < 26; ++i) {
```

```
            input >> weights[i];
```

```
        }
```

```
        input.close();
```

```
        std::string s_out(s.size(), ' ');
```

```
        std::string s_double;
```

```
        for (char ch = 'a'; ch <= 'z'; ch++) {
```

```
            size_t first = s.find(ch);
```

```
            if (first != std::string::npos) {
```

```
                size_t last = s.rfind(ch);
```

```
                if (first != last) {
```

```
                    s_double += ch;
```

```
                    s[first] = ' ';
```

```

        s[last] = ' ';
    }
}
}
sort(s_double.begin(), s_double.end(), compare_weights);
for (size_t i = 0, j = s_out.size() - 1; i < s_double.size(); i++, j--) {
    s_out[i] = s_double[i];
    s_out[j] = s_double[i];
}
for (size_t i = 0, n = s_double.size(); i < s.size(); i++) {
    if (s[i] != ' ') {
        s_out[n++] = s[i];
    }
}
std::ofstream output("aurora.out");
if (output.is_open()) {
    output << s_out;
    output.close();
} else {
    std::cout << "input aurora.in not found!" << std::endl;
}
return 0;
}

```

Пояснение к примененному алгоритму:

- 1) Считываем строку и веса букв;
- 2) Находим для каждой буквы, встречающейся ≥ 2 раза, крайние вхождения, помечаем их позиции как использованные и добавляем букву один раз в список парных символов;
- 3) Сортируем список парных символов по убыванию веса;
- 4) Размещаем парные символы симметрично: самый «тяжёлый» — в начало и конец, следующий — внутрь.

- 5) Оставшиеся символы (включая внутренние вхождения парных букв) размещаем в центре в том порядке, в котором они встречались в исходной строке.

Задача №Н «Магазин»

```
#include <algorithm>
```

```
#include <fstream>
```

```
#include <iostream>
```

```
int main() {
```

```
    int n;
```

```
    int k;
```

```
    std::ifstream input("shop.in");
```

```
    if (input.is_open()) {
```

```
        if (input >> n && input >> k && n >= 1 && n <= 100000 && k >= 2 && k <= 100) {
```

```
            int* prices = new int[n];
```

```
            for (int i = 0; i < n; i++) {
```

```
                input >> prices[i];
```

```
            }
```

```
            std::sort(prices, prices + n);
```

```
            int sum = 0;
```

```
            for (int i = n - 1; i >= 0; --i) {
```

```
                if ((n - i) % k != 0) {
```

```
                    sum += prices[i];
```

```
                }
```

```
            }
```

```
            delete[] prices;
```

```
            std::ofstream output("shop.out");
```

```
            if (output.is_open()) {
```

```
                output << sum;
```

```
                output.close();
```

```
            }
```

```
        } else {
```

```
            std::cout << "n k read error!" << std::endl;
```

```
}  
input.close();  
} else {  
    std::cout << "input shop.in not found!" << std::endl;  
}  
return 0;  
}
```

Пояснение к примененному алгоритму:

- 1) Сортируем все товары по возрастанию цен;
- 2) Рассматриваем товары с конца, начиная с самых дорогих;
- 3) Каждый k-ый товар от самого дорогого цен считается бесплатным и не добавляется к сумме;
- 4) Суммируем цены всех товаров и записываем результат в выходной файл.